Repcard CRM Integration

1. Overview

1.1 Description

Repcard CRM Integration connects **Repcard CRM** (field sales CRM for solar, home services, and insurance) with **Newo Platform**.

It enables:

- Creating appointments in Repcard with customer details, address, and calendar selection
- Cancelling existing appointments by updating their status
- Checking availability by computing free 30-minute slots from existing appointments
- Creating contacts (customers) in Repcard with type, owner, and status assignment
- Sending digital business cards (RepCards) to contacts via SMS or email
- Dynamic calendar synchronization from the Repcard API at setup
- DevMode for testing without real API credentials (LLM-emulated responses)
- Auto-configuring conversation canvas with booking and cancellation scenarios

This integration is designed for **field sales teams** (solar installers, home services providers, insurance agents) who use Repcard CRM to manage their leads and appointments and need **an AI receptionist to book, cancel, and check availability** through voice or chat.

1.2 Use Cases

- When a caller requests an appointment --> Agent collects details (name, phone, email, address, date/time) and creates an appointment in Repcard
- When a caller wants to cancel an appointment --> Agent finds the booking and cancels it in Repcard
- When a caller asks about availability --> Agent fetches existing appointments and computes free 30-minute slots over the next 5 days
- When a contact needs to be created --> Agent collects customer info and creates a contact in Repcard via POST /customers
- When a digital business card needs to be sent --> Agent sends a RepCard to a contact via SMS or email
- When the project is published --> Agent syncs available calendars from Repcard API and populates the calendar dropdown

1.3 Supported Features

Feature	Supported	Notes
Create Appointment	Yes	Full customer details, address, calendar selection, setter email
Cancel Appointment	Yes	`PATCH /appointments/{id}` with cancellation notes
Check Availability	Yes	Computes free 30-min slots from existing appointments (5-day window)
Create Contact	Yes	POST /customers with type, owner, status
Send RepCard	Yes	POST /send-repcard (digital business card via SMS/email)

Calendar Sync	Yes	Dynamic sync from GET /calendar/lists on publish
Canvas Auto-Setup	Yes	Booking + cancellation scenarios and intents
DevMode (LLM Emulation)	Yes	Test without real API -- LLM generates realistic responses
Reschedule	No	Cancel + rebook as workaround
Existing Client Lookup	No	Not implemented

2. Requirements

Before installation:

- Active **Repcard CRM** Enterprise account with API access
- **API Key** from Repcard Company Settings > API Keys tab
- At least one calendar configured in Repcard
- A setter (sales rep) email address for appointment assignment
- A contact owner email for contact creation

3. How to Setup

3.1 Step 1 -- Get API Credentials from Repcard CRM

1. Log in to your **Repcard CRM** account
2. Navigate to **Company Settings** > **API Keys** tab
3. Generate or copy your **API Key**
4. Note: API access requires an **Enterprise** account
5. Note your **Calendar IDs** (these will be synced automatically on publish)
6. Note the **setter email** address for appointment assignment

3.2 Step 2 -- Connect in Platform

1. Open **Newo --> Projects**
2. Set the following attributes in **Builder / Attributes**:

Attribute	Required	Default	Description
`repcard_api_key`	Yes	`--`	API Key from Repcard Company Settings > API Keys tab
`repcard_base_url`	No	`https://api.repcard.com`	API base URL
`repcard_mode`	No	`Production`	`Production` for real API, `DevMode` for LLM emulation
`repcard_enable_booking`	No	`True`	Enable appointment creation
`repcard_enable_cancellation`	No	`True`	Enable appointment cancellation

`repcard_enable_contact_creation`	No	`True`	Enable contact creation
`repcard_enable_send_repcard`	No	`True`	Enable sending digital business cards
`repcard_default_calendar`	No	(auto-synced)	Calendar ID for new appointments (dropdown populated after sync)
`repcard_default_setter`	Yes*	`--`	Setter (sales rep) email for new appointments
`repcard_default_contact_type`	No	`Lead`	Contact type for new contacts (Lead, Customer, Recruit)
`repcard_default_contact_owner`	Yes*	`--`	Owner (assigned rep) email for new contacts
`repcard_default_contact_status`	No	`New`	Status for new contacts
`repcard_default_repcard_id`	No	`--`	Digital business card template ID
`repcard_setup_scenarios`	No	`True`	Auto-add booking/cancellation scenarios to canvas

*Required for the respective feature to work (booking requires setter, contacts require owner).

3. Click **Save + Publish All**
4. On publish, the InitFlow automatically:
    - Creates HTTP connector for Repcard API (`repcard_connector`)
    - Sets all customer attributes with metadata
    - Registers booking, availability, and cancellation tools on ConvoAgent
    - Syncs calendars from Repcard API (GET /calendar/lists)
    - Populates the calendar dropdown with synced values

4. How to Use

4.1 How the Integration Works

The integration uses the Repcard **REST API** with `x-api-key` header authentication. All requests go through the centralized NetworkFlow, which routes to either the real HTTP connector (Production) or LLM emulation (DevMode).

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Repcard CRM

Available Triggers

Trigger	Event	Description
Create Appointment	`book_slot`	When the agent confirms a booking

Cancel Appointment	`convo_agent_cancel_booking`	When the agent processes a cancellation
Check Availability	`get_available_slots`	When the agent checks for open time slots
Create Contact	`repcard_create_contact`	When a new contact needs to be created
Send RepCard	`repcard_send_repcard`	When a digital business card is sent
Setup	`project_publish_finish`	Initialize integration on deploy
Canvas Setup	`gm_workflow_builder_canvas_built`	Auto-add scenarios to canvas

Available Actions

- **Create Appointment** -- Submit new appointment (POST `/appointments`)
- **Cancel Appointment** -- Cancel appointment with notes (PATCH `/appointments/{id}`)
- **Get Appointments** -- Fetch existing appointments for availability calculation (GET `/appointments?from_date&to_date`)
- **Create Contact** -- Create new customer (POST `/customers`)
- **Send RepCard** -- Send digital business card (POST `/send-repcard`)
- **Sync Calendars** -- Fetch active calendars (GET `/calendar/lists?status=active`)

4.2 Flows

Flow	Purpose
InitFlow	Initialize integration: connectors, attributes, calendar sync
NetworkFlow	HTTP execution layer with Production/DevMode routing
ApiFlow	Centralized request builders + result routing for all domain API calls
BookingFlow	Entry point for booking -- validates and delegates to ApiFlow
CancellationFlow	Entry point for cancellation -- validates and delegates to ApiFlow
AvailabilityFlow	Entry point for availability -- delegates to ApiFlow, computes free slots
ContactFlow	Entry point for contact creation -- validates and delegates to ApiFlow
RepcardSendFlow	Entry point for sending RepCards -- validates and delegates to ApiFlow
CanvasSetupFlow	Auto-add booking + cancellation scenarios to canvas
LogFlow	Centralized debug logging

4.3 Architecture Diagrams

Overall Sequence

```
sequenceDiagram
    participant U as "User"
    participant A as "ConvoAgent"
    participant I as "Repcard Integration"
```

```
    participant API as "Repcard API"

    Note over I,API: "Setup (on publish)"
    A->>I: "project_publish_finish"
    I->>I: "Create connector, set attributes"
    I->>API: "GET /calendar/lists?status=active"
    API-->>I: "Calendar list"
    I->>I: "Store calendars, populate dropdown"

    Note over U,API: "Check Availability"
    U->>A: "What times are available?"
    A->>I: "get_available_slots"
    I->>API: "GET /appointments?from_date&to_date"
    API-->>I: "Existing appointments"
    I->>I: "Compute free 30-min slots"
    I->>A: "result_success {available_slots}"
    A->>U: "Here are the available times..."

    Note over U,API: "Booking"
    U->>A: "Book me for Tuesday at 10am"
    A->>A: "Collect: name, phone, email, address, date/time"
    A->>I: "book_slot {customerInfo, bookingDateTime, calendarId}"
    I->>API: "POST /appointments"
    API-->>I: "{id, status}"
    I->>A: "result_success {booking_id}"
    A->>U: "Your appointment is confirmed!"

    Note over U,API: "Cancellation"
    U->>A: "Cancel my appointment"
    A->>I: "convo_agent_cancel_booking {booking_id}"
    I->>API: "PATCH /appointments/{id}"
    API-->>I: "success"
    I->>A: "result_success"
    A->>U: "Your appointment has been cancelled"
```

InitFlow

```
flowchart TD
    E["project_publish_finish"] --> CHECK{"target_reference
==\nrepcard_integration?"}
    CHECK -->|"No"| SKIP["Return"]
    CHECK -->|"Yes"| CALS_RAW["Load repcard_calendars\nfrom attributes"]
    CALS_RAW --> BUILD_ENUM["Build calendar enum\nfor booking schema"]
    BUILD_ENUM --> SET_SA["Set superagent attributes:\nbooking,
availability,\ncancellation enabled"]
    SET_SA --> SET_SCHEMA["Set booking payload\nschema with calendarId enum"]
    SET_SCHEMA --> SET_PA["Set project attributes:\nmode, api_key, base_url,\nenable
flags, defaults"]
    SET_PA --> CONNECTOR["Create repcard_connector\n(HTTP integration)"]
    CONNECTOR --> PERSONA["Create SetupPersona\nfor background tasks"]
    PERSONA --> KEY_CHECK{"repcard_api_key\nconfigured?"}
```

```
    KEY_CHECK -->|"No"| DONE["Done"]
    KEY_CHECK -->|"Yes"| SYNC["Send repcard_sync_calendars_request"]
    SYNC --> DONE
```

NetworkFlow -- Production vs DevMode

```
flowchart TD
    REQ["repcard_execute_request"] --> MODE{"repcard_mode?"}
    MODE -->|"Production"| REAL["_executeRequestSkill\nBuild URL, set x-api-key
header"]
    MODE -->|"DevMode"| EMULATE["_emulateRequestSkill\nGen() LLM
generates\nrealistic response"]
    REAL --> CONNECTOR["SendCommand to\nrepcard_connector"]
    CONNECTOR --> RESP_OK["result_success\nfrom HTTP connector"]
    CONNECTOR --> RESP_ERR["result_error\nfrom HTTP connector"]
    RESP_OK --> STATUS{"statusCode\n200 or 201?"}
    STATUS -->|"Yes"| SUCCESS["repcard_result_success"]
    STATUS -->|"No"| ERROR["repcard_result_error"]
    RESP_ERR --> ERROR
    EMULATE --> SUCCESS
```

ApiFlow

```
flowchart TD
    subgraph "Request Builders"
        CA["CreateAppointmentApiSkill\nPOST /appointments"]
        CC["CreateContactApiSkill\nPOST /customers"]
        SR["SendRepcardApiSkill\nPOST /send-repcard"]
        CANCEL["CancelAppointmentApiSkill\nPATCH /appointments/{id}"]
        AVAIL["GetAvailableSlotsApiSkill\nGET /appointments"]
        SYNC["SyncCalendarsApiSkill\nGET /calendar/lists"]
    end

    subgraph "Result Router"
        ERS["ExecuteResultSuccessSkill\nRoute by runSkill"]
        ERE["ExecuteResultErrorSkill\nRoute by errorSkill"]
    end

    subgraph "Success Handlers"
        CAS["_createAppointmentSuccessSkill\nemit repcard_booking_success"]
        CCS["_createContactSuccessSkill\nemit repcard_contact_success"]
        SRS["_sendRepcardSuccessSkill\nemit repcard_send_repcard_success"]
        CANS["_cancelAppointmentSuccessSkill\nemit repcard_cancellation_success"]
        BAS["_buildAvailableSlotsSkill\ncompute free slots,\nemit
repcard_availability_success"]
        SCS["_syncCalendarsSuccessSkill\nstore calendars,\nupdate dropdown"]
    end

    CA --> |"repcard_execute_request"| ERS
    CC --> |"repcard_execute_request"| ERS
```

```
    SR --> |"repcard_execute_request"| ERS
    CANCEL --> |"repcard_execute_request"| ERS
    AVAIL --> |"repcard_execute_request"| ERS
    SYNC --> |"repcard_execute_request"| ERS

    ERS --> |"runSkill routing"| CAS
    ERS --> |"runSkill routing"| CCS
    ERS --> |"runSkill routing"| SRS
    ERS --> |"runSkill routing"| CANS
    ERS --> |"runSkill routing"| BAS
    ERS --> |"runSkill routing"| SCS

    ERE --> |"errorSkill routing"| ERR_HANDLERS["Error emit skills\n(per domain)"]
```

BookingFlow

```
flowchart TD
    E["book_slot"] --> GATE{"repcard_enable_booking\n== True?"}
    GATE -->|"No"| SKIP["Return"]
    GATE -->|"Yes"| DELEGATE["Send repcard_book_slot_request\nto ApiFlow"]
    DELEGATE --> API["CreateAppointmentApiSkill"]
    API --> VALIDATE{"api_key + firstName +\nphone + bookingDateTime +\ncalendarId +
setter?"}
    VALIDATE -->|"No"| ERR["repcard_booking_error"]
    VALIDATE -->|"Yes"| BUILD["Build POST /appointments body:\ncalendarId,
setterEmail,\nstartAt, endAt, timezone,\ncustomer {name, phone, email, address}"]
    BUILD --> EXEC["repcard_execute_request"]
    EXEC --> OK{"Success?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| EXTRACT["Extract appointment ID\nfrom response"]
    EXTRACT --> SUCCESS["repcard_booking_success\n{booking_id, bookingDateTime}"]
    SUCCESS --> RESULT["result_success\ntargetAction: book_slot"]
```

CancellationFlow

```
flowchart TD
    E["convo_agent_cancel_booking"] --> GATE{"repcard_enable_cancellation\n==
True?"}
    GATE -->|"No"| SKIP["Return"]
    GATE -->|"Yes"| DELEGATE["Send repcard_cancel_booking_request\nto ApiFlow"]
    DELEGATE --> API["CancelAppointmentApiSkill"]
    API --> VALIDATE{"api_key +\nbooking_id?"}
    VALIDATE -->|"No"| ERR["repcard_cancellation_error"]
    VALIDATE -->|"Yes"| BUILD["Build PATCH /appointments/{id}\nbody: {notes}"]
    BUILD --> EXEC["repcard_execute_request"]
    EXEC --> OK{"Success?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| SUCCESS["repcard_cancellation_success\n{booking_id}"]
    SUCCESS --> RESULT["result_success\ntargetAction: cancel_booking"]
```

AvailabilityFlow

```
flowchart TD
    E["get_available_slots"] --> DELEGATE["Send
repcard_check_availability_request\nto ApiFlow"]
    DELEGATE --> API["GetAvailableSlotsApiSkill"]
    API --> VALIDATE{"api_key\nconfigured?"}
    VALIDATE -->|"No"| ERR["repcard_availability_error"]
    VALIDATE -->|"Yes"| BUILD["Build GET /appointments\nquery:
from_date=today,\nto_date=today+5, per_page=100"]
    BUILD --> EXEC["repcard_execute_request"]
    EXEC --> OK{"Success?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| SLOTS["_buildAvailableSlotsSkill:\nParse busy times from
appointments\nGenerate 30-min slots (9am-5pm)\nExclude busy + past slots\nReturn top
20"]
    SLOTS --> SUCCESS["repcard_availability_success\n{available_slots,
total_count}"]
    SUCCESS --> RESULT["result_success\ntargetAction: get_available_slots"]
```

ContactFlow

```
flowchart TD
    E["repcard_create_contact"] --> GATE{"repcard_enable_contact_creation\n==
True?"}
    GATE -->|"No"| SKIP["Return"]
    GATE -->|"Yes"| DELEGATE["Send repcard_create_contact_request\nto ApiFlow"]
    DELEGATE --> API["CreateContactApiSkill"]
    API --> VALIDATE{"api_key + firstName +\nphone + owner?"}
    VALIDATE -->|"No"| ERR["repcard_contact_error"]
    VALIDATE -->|"Yes"| BUILD["Build POST /customers body:\nfirstName, lastName,
userEmail,\ntype, statusName, email,\nphoneNumber, contactSource=AI"]
    BUILD --> EXEC["repcard_execute_request"]
    EXEC --> OK{"Success?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| SUCCESS["repcard_contact_success"]
```

RepcardSendFlow

```
flowchart TD
    E["repcard_send_repcard"] --> GATE{"repcard_enable_send_repcard\n== True?"}
    GATE -->|"No"| SKIP["Return"]
    GATE -->|"Yes"| DELEGATE["Send repcard_send_repcard_request\nto ApiFlow"]
    DELEGATE --> API["SendRepcardApiSkill"]
    API --> VALIDATE{"api_key + contact_id +\nrepcard_id?"}
    VALIDATE -->|"No"| ERR["repcard_send_repcard_error"]
    VALIDATE -->|"Yes"| BUILD["Build POST /send-repcard body:\nexisting_contact,
sms_or_email,\nrepcard, message"]
    BUILD --> EXEC["repcard_execute_request"]
```

```
    EXEC --> OK{"Success?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| SUCCESS["repcard_send_repcard_success"]
```

CanvasSetupFlow

```
flowchart TD
    E["gm_workflow_builder_canvas_built"] --> CHECK{"repcard_setup_scenarios\n==
False?"}
    CHECK -->|"Yes"| SKIP["Return"]
    CHECK -->|"No"| LIB["SetupLibrarySkill"]
    LIB --> ITEMS["Define items to insert:\n- cancellation_via_agent_intent\n-
library_intent_types_common_appointment_agent\n- cancellation_via_agent_scenario\n-
repcard_schedule_appointment_via_agent_scenario"]
    ITEMS --> LOOP["For each item:\nCheck if already on canvas\nIf not, get from
_getScenariosSkill\nor library"]
    LOOP --> DELETE["Delete old generic
items:\nscenario_home_service_appointment_request\nintent_type_appointment_scheduling\n
    DELETE --> ADD["Add Repcard-specific items\nto canvas via batch"]
```

LogFlow

```
flowchart TD
    E["repcard_log"] --> LOG["LogSkill:\nDUMMY(Repcard: message)"]
```

4.4 Canvas Scenarios

When `repcard_setup_scenarios` is `True` (default), the CanvasSetupFlow automatically adds:

Scenarios:

- **"Scheduling Appointment via Agent"** (`repcard_schedule_appointment_via_agent_scenario`) -- 10-step guided booking flow that collects all required fields (name, phone, email, address, ZIP, date/time) with code-phrases
- **"Canceling Appointment via Agent"** (`cancellation_via_agent_scenario`) -- 6-step cancellation flow with confirmation and code-phrases

Intent Types:

- **"[L] Appointment via Agent"** (`library_intent_types_common_appointment_agent`) -- from library
- **"[T] Cancellation via Agent"** (`cancellation_via_agent_intent`) -- caller wants to cancel

4.5 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice).

DevMode: Set `repcard_mode` to `DevMode` to test without real API credentials. The integration will use LLM to generate realistic Repcard API responses.

4.6 Testing and Verification

To test the integration:

1. **Setup**: Publish the project and verify calendar sync completed (check `repcard_calendars` attribute and `repcard_default_calendar` dropdown)
2. **Availability**: Ask about available times -- verify agent returns computed free slots
3. **Booking**: Request an appointment -- provide name, phone, email, address, and date --> verify appointment created in Repcard
4. **Cancellation**: Ask to cancel -- verify appointment status updated in Repcard
5. **Contact Creation**: Trigger contact creation -- verify customer created in Repcard via POST /customers

If no action occurs:

- Ensure API key is set (`repcard_api_key`) -- requires Enterprise account
- Verify `repcard_mode` is `Production` (not `DevMode`)
- Check feature flags (`repcard_enable_booking` , `repcard_enable_cancellation` , etc.)
- Confirm `repcard_default_setter` is set (required for booking)
- Confirm `repcard_default_contact_owner` is set (required for contacts)
- Verify calendars synced (check `repcard_default_calendar` dropdown)
- Re-publish after configuration changes

> *Note: This integration creates real appointments and contacts in Repcard CRM. Changes made through the agent are reflected in the live system.*

5. FAQ

Q: How does authentication work? A: Repcard uses an **API Key** sent in the `x-api-key` HTTP header. API access requires an Enterprise-tier Repcard CRM account. The key is obtained from Company Settings > API Keys tab.

Q: What is DevMode? A: When `repcard_mode` is set to `DevMode` , the integration uses LLM (Gen) to emulate Repcard API responses instead of making real HTTP requests. Useful for testing the booking flow without creating real appointments.

Q: How does calendar selection work? A: On publish, the integration calls `GET /calendar/lists?status=active` to fetch all active calendars. These are stored in `repcard_calendars` and populate the `repcard_default_calendar` dropdown. The LLM can also select a `calendarId` from the enum in the booking schema. If no calendar is selected, the default is used.

Q: Why is the customers endpoint POST /customers and not POST /contacts? A: Repcard CRM uses `/customers` as the endpoint name for creating contacts. This is the official Repcard API convention despite the feature being called "contact creation" in the integration.

Q: How does availability checking work? A: The integration fetches all existing appointments for the next 5 days via `GET /appointments?from_date&to_date` , then computes free 30-minute slots during business hours (9:00 AM to 5:00 PM) by excluding busy time slots. Past slots are also excluded. Up to 20 available slots are returned.

Q: What DateTime format does the API use? A: The Repcard API uses `YYYY-MM-DD HH:MM:SS` format. The integration automatically appends `:00` seconds if the input is in `YYYY-MM-DD HH:MM` format (16 characters).

Q: How does the calendarId work? A: The `calendarId` is numeric and identifies a calendar in Repcard CRM. It is synced dynamically from the API. The `setterEmail` identifies the sales representative assigned

to the appointment.

Q: Can I use the Send RepCard feature through the conversational agent? A: The Send RepCard feature (`POST /send-repcard`) is primarily designed for Zapier-style automation triggers. It requires a `contact_id` and a `repcard_default_repcard_id` template to be configured.